

ADVANCING
HUMANITY



Kampus
Merdeka
INDONESIA, JAYA

ANSWER SHEET

DAC 2023

PRELIMINARY ROUND



DATA ANALYSIS COMPETITION 2023

TEAM NAME

Cola Bee

TEAM ID

DAC-01-0036

UNIVERSITY

BINA NUSANTARA

DATA ANALYSIS COMPETITION 2023
INSTITUT TEKNOLOGI SEPULUH NOPEMBER

CHAPTER I: Introduction

In the realm of network security, the rising concern centers on various types of network attacks. The primary objective is to detect these attacks based on network usage data. In an increasingly interconnected digital world, these attacks can range from Distributed Denial of Service (DDoS) assaults to sophisticated malware intrusions. To counter these threats, the focus is on analyzing network usage data patterns and anomalies. By leveraging advanced machine learning and artificial intelligence models, security experts aim to differentiate normal network behavior from suspicious activities and respond swiftly to potential attacks.

CHAPTER II: Theoretical Framework

A. The Random Forest Algorithm

The Random Forest Algorithm is a machine learning algorithm that works by creating multiple randomly generated decision trees with low correlations to form a random forest. The core techniques and principles that enable the Random Forest Algorithm to work are as follows:

a. Ensemble Technique

The Random Forest Algorithm itself is an ensemble learning technique, which combines multiple individual models to achieve more accurate and robust predictions.

b. Decision Trees

Decision Trees are the heart of the Random Forest Algorithm. Decision trees make predictions by recursively splitting the dataset into subsets based on the values of variables.

The Random Forest Algorithm itself is a collection of decision trees, and each tree in the ensemble is a subset of the training dataset with replacement, referred to as a bootstrap sample. From this subset of training data, one-third of it is set aside to become test data, known as an out-of-bag sample (OOB). After that, another instance of randomness is introduced by using feature bagging to add more diversity among the decision trees, thereby reducing the correlation between each decision tree.

B. Correlation

Correlation among variables is a core concept in statistical analysis that explores the relationships between different variables. It is crucial in understanding how changes in one variable are associated with changes in another, offering insights into potential connections and dependencies. Various methods are employed to measure the correlation between variables in this analysis, including:

a. Pearson Correlation

Pearson Correlation assesses the linear relationship between two continuous variables. The resulting correlation coefficient, denoted as "r," quantifies the strength and direction of this linear association. It ranges from -1 (perfect negative correlation) to 1 (perfect positive correlation), with 0 indicating no linear relationship. This method is valuable for understanding the degree of connection between variables.

b. Chi-Square Test

The Chi-Square Test examines the association between two categorical variables. It compares observed and expected frequency distributions to determine if there is a statistically significant difference. This test helps ascertain if variables are dependent or independent, aiding in further exploration of their relationships.

c. ANOVA (Analysis of Variance)

ANOVA analyses variations between groups within a categorical variable concerning a continuous variable. Its primary purpose is to identify statistically significant differences in means among these groups. ANOVA enables the comparison of multiple groups, identifying those with distinct characteristics.

d. Cramer's V

Cramer's V extends the chi-square test to measure the strength of association between two categorical variables. It quantifies the effect size of the relationship, ranging from 0 (no association) to 1 (strong association). Cramer's V assists in interpreting the significance of relationships between categorical variables.

CHAPTER III: Analytical Steps

To analyze the data, we are using the steps and principles of Exploratory Data Analysis (EDA), data preparation for the dataset, modeling, and prediction. Exploratory data analysis itself consists of describing the dataset and its variables, identifying missing values in the dataset, identifying correlated variables, and detecting extremes or anomalies in the dataset.

A. Identifying the Basic Description of the Dataset and Variables

We can obtain the basic description of the variables from the provided guidebook. Additionally, we can derive the general characteristics of the dataset using the "basicSummary()" function that we have created. This function helps us identify the most frequently occurring values, how many times each value occurs, and the count of missing values within a variable. We can also view the descriptive statistics for each variable by utilizing the "describe" function from the Pandas library.

B. Data Preparation

In this step, we adjust the data types of each variable to their respective correct data types, in accordance with the values and descriptions provided in the guidebook. Additionally, we perform data cleaning by replacing all missing values in categorical data with their mode and in numerical data with their median.

C. Visualization and Correlation Checking

First, the data was meticulously prepared and categorized into two main types: numeric and categorical. The analysis then commenced with the examination of correlations between categorical variables. This was achieved using the chi-square test, which assessed the association between different pairs of categorical variables. Contingency tables were created for each pair, and the chi-square statistic along with its corresponding p-value were calculated to determine the strength of association.

Moving forward, the focus shifted to investigating how categorical variables related to numeric ones. To accomplish this, an analysis of variance (ANOVA) was performed. ANOVA allowed for the comparison of means across different categories of a categorical variable concerning numeric variables. Conversely, the analysis also explored how numeric variables related to the categorical target variable. This involved another application of ANOVA, which assessed whether the means of numeric variables significantly differed across various categories of the target variable. The outcomes of these analyses were instrumental in variable selection. A dictionary was constructed to store computed p-values for both categorical and numeric variables. Variables exhibiting p-values below the 0.05 significance threshold were considered significant, indicating a substantial correlation with the target variable.

To provide a more intuitive understanding of the findings, data visualization played a vital role. For categorical variables, count plots were employed to visualize their distribution across different attack types. In contrast, box plots were used for numeric variables, with the flexibility to apply a logarithmic scale when required. These visualizations enabled the observation of how variable distributions varied concerning different attack types. Additionally, Cramer's V was applied to evaluate the association between the target variable, "type_of_attack," and the categorical variable, "protocol_type." Cramer's V, a measure of association for categorical variables, provided insights into the strength and direction of the relationship between these two variables.

D. Modelling

In this step, we model the dataset using the Random Forest algorithm, as it has been identified as one of the most accurate machine learning models for detecting network anomalies. To use the Random Forest algorithm, we first need to preprocess the data by encoding the categorical variables in the dataset. Then, we proceed to split the dataset into training and testing data subsets. After completing the data

preprocessing, we feed the training data into the Random Forest algorithm to create the model.

E. Prediction

In this step, we import the prediction dataset provided and encode all of the categorical variables. Additionally, we drop the 'id' column and employ the previously created model to predict the 'type_of_attack' based on the data provided in the dataset.

CHAPTER IV: Analysis of Results

From the analytics conducted on the dataset, we obtained several key findings. The key findings will be segmented into dataset description, correlation findings, and model accuracy.

A. Dataset Description

The dataset comprises 112,446 objects and 42 variables, with a maximum of 42,430 missing values in a single variable. There are 4 categorical variables, and 38 others are numerical variables. The variable that serves as the target variable here is the 'type_of_attack' variable, which has 8 unique values. These unique values include nmap, neptune, normal, Denial of Service Attack, portsweep, satan, ipsweep, and smurf. The mode of the 'type_of_attack' variable is 'normal,' which appears 22,492 times.

B. Correlation Findings

During the correlation analysis between each variable and the 'type_of_attack' variable, we identified 17 significant variables with p-values ranging from greater than 0 to 0.05. These variables include: duration, land, wrong_fragment, hot, num_failed_logins, num_compromised, root_shell, su_attempted, num_root, num_file_creations, num_shells, num_access_files, is_guest_login, diff_srv_rate, srv_diff_host_rate, dst_host_same_src_port_rate, and dst_host_srv_diff_host_rate.

Additionally, there are 18 variables with p-values equal to 0, namely: logged_in, count, error_rate, srv_error_rate, rerror_rate, srv_error_rate, same_srv_rate, dst_host_count, dst_host_srv_count, dst_host_same_srv_rate, dst_host_diff_srv_rate, dst_host_error_rate, dst_host_srv_error_rate, dst_host_error_rate, dst_host_srv_error_rate, protocol_type, service, and flag. These small p-values indicate a robust relationship between these variables and the "type_of_attack" variable.

In addition, we have identified several strong correlations among numerical variables (with correlations greater than 0.80). These correlations occur in the following variables: hot and is_guest_login, dst_host_srv_count and dst_host_same_srv_rate, error_rate and dst_host_error_rate, srv_error_rate and dst_host_error_rate, error_rate and dst_host_srv_error_rate, num_root and num_compromised,

error_rate and dst_host_error_rate, srv_error_rate and dst_host_error_rate, dst_host_srv_error_rate and dst_host_error_rate, error_rate and dst_host_srv_error_rate, srv_error_rate and dst_host_srv_error_rate, srv_error_rate and dst_host_srv_error_rate.

C. Model

From the correlation findings we have concluded to make 2 models. 1st Model will be using all independent variables as the parameters. The 2nd Model will be using only independent variables with p-values of 0. From the testing that we have done we have concluded that the 1st Model is more accurate than the second model with an accuracy of 0.994. With one of the trees as such



Figure 1.1. Random Forest Decision Tree

From the decision tree above we can see that even only 1 of the decision trees is so complicated and robust because of the number of variables and objects in the dataset and that is part of the reason we decided to use random forest algorithms instead of decision tree.

CHAPTER V : Conclusion and Recommendation

The conclusion of this report is that the data given is quite a messy data with a lot of missing values. But with data preparation techniques we can substitute the missing values with the median and mode for the respective variables data type. From the correlation checking result we have seen that all of the independent variables are closely related to the dependent variables and would be wise to be put as parameters for modelling later on. On the modelling we have seen that using all the independent variables gave more accurate results than only using independent variables with p-values of 0.

REFERENCES

- [1] IBM, “What is Random Forest? | IBM,” *www.ibm.com*, 2023.
<https://www.ibm.com/topics/random-forest>
- [2] K. A. Dhanya, S. Vajipayajula, K. Srinivasan, A. Tibrewal, T. S. Kumar, and T. G. Kumar, “Detection of Network Attacks using Machine Learning and Deep Learning Models,” *Procedia Computer Science*, vol. 218, pp. 57–66, 2023, doi: <https://doi.org/10.1016/j.procs.2022.12.401>.
- [3] N. D. Jespersen, J. E. Brady och A. Hyslop, *Chemistry: The Molecular Nature of Matter*, Hoboken: John Wiley & Sons, Inc, 2012.
- [4] L. J. Malone och T. Dolter, *Basic Concepts of Chemistry*, New York: John Wiley & Sons, Inc, 2010.
- [5] A. Field, “*Discovering Statistics Using IBM SPSS Statistics*”, Sage Publications, 2018.

SYNTAX

```

import pandas as pd
import numpy as np
import matplotlib.pyplot as plt
import seaborn as sns
from scipy import stats

def BasicSummary(df, dgts=3):
    m = df.shape[1]
    varNames = df.columns.tolist()
    varType = [''] * m
    topLevel = [''] * m
    topCount = [0] * m
    missCount = [0] * m
    levels = [0] * m

    for i in range(m):
        x = df.iloc[:, i]
        varType[i] = x.dtype.name
        xtab = pd.Series(x).value_counts(dropna=False)
        levels[i] = len(xtab)
        nums = xtab.values
        maxnum = np.max(nums)
        topCount[i] = maxnum
        maxIndex = np.argmax(nums)
        lvls = xtab.index.tolist()
        topLevel[i] = lvls[maxIndex]
        missIndex = np.where((pd.isnull(x)) | (x == '') |
(x == ' '))[0]
        missCount[i] = len(missIndex)

    n = df.shape[0]
    topFrac = np.round(np.array(topCount) / n)
    missFrac = np.round(np.array(missCount) / n)

    summaryFrame = pd.DataFrame({
        'variable': varNames,
        'type': varType,
        'levels': levels,
        'topLevel': topLevel,
        'topCount': topCount,
        'topFrac': topFrac,
        'missFreq': missCount,

```




NIDYA (+62 813-3364-7263) • IQBAL (+62 812-5220-5874)

```

# Change Data Type of the respetive columns
newTrain = newTrain.astype({"land": object,
                             "logged_in": object,
                             "root_shell": object,
                             "su_attempted": object,
                             "is_host_login": object,
                             "is_guest_login": object,
                             "dst_host_same_src_port_rate":
float,
                             "dst_host_rerror_rate": float})

newTrain.describe()

"""Fill Missing value with mode for categorical and median
for numerical"""

for col in newTrain.columns:
    if newTrain[col].dtype == 'object': # categorical
column
        newTrain[col] =
newTrain[col].fillna(newTrain[col].mode()[0])
    else: # numerical column
        newTrain[col] =
newTrain[col].fillna(newTrain[col].median())

BasicSummary(newTrain)

"""# Visualization & Correlation Check"""

correlation_matrix = newTrain.corr()
correlation_matrix

def distinguish_variable_types(dataframe):
    numerik = [] # Ini akan berisi nama kolom variabel
numerik
    kategorikal = [] # Ini akan berisi nama kolom variabel
kategorikal

    for column in dataframe.columns:
        if
pd.api.types.is_numeric_dtype(dataframe[column]):
            numerik.append(column)

```

```

elif
pd.api.types.is_categorical_dtype(dataframe[column]) or
pd.api.types.is_string_dtype(dataframe[column]):
    kategorikal.append(column)

return numerik, kategorikal

# Panggil fungsi distinguish_variable_types
numerik, kategorikal = distinguish_variable_types(newTrain)
print("Variabel Numerik:", numerik)
print("Variabel Kategorikal:", kategorikal)

target_variable = 'type_of_attack'

# Pisahkan variabel numerik dan kategorikal
numerik, kategorikal = distinguish_variable_types(newTrain)

# Hitung korelasi antara variabel kategorikal dengan
variabel kategorikal (gunakan uji chi-square)
for cat_var in kategorikal:
    if cat_var != target_variable:
        contingency_table =
pd.crosstab(newTrain[target_variable], newTrain[cat_var])
        chi2, p, _, _ =
stats.chi2_contingency(contingency_table)
        print(f"Korelasi antara {target_variable} dan
{cat_var}: p-value = {p}")

# Hitung korelasi antara variabel kategorikal dengan
variabel numerik (gunakan uji ANOVA)
for num_var in numerik:
    f_statistic, p_value =
stats.f_oneway(newTrain[newTrain[target_variable] ==
'normal'][num_var], newTrain[newTrain[target_variable] !=
'normal'][num_var])
    print(f"Korelasi antara {target_variable} dan
{num_var}: p-value = {p_value}")

# Periksa missing values dalam DataFrame
missing_values = newTrain[numerik].isnull().sum()
print(missing_values)

```

```

for num_var in numerik:
    f_statistic, p_value =
stats.f_oneway(newTrain[newTrain[target_variable] ==
'normal'][num_var], newTrain[newTrain[target_variable] !=
'normal'][num_var])
    print(f"Korelasi antara {target_variable} dan
{num_var}: p-value = {p_value}")

target_variable = 'type_of_attack'

# Pisahkan variabel numerik dan kategorikal
numerik, kategorikal = distinguish_variable_types(newTrain)

# Buat dictionary untuk menyimpan hasil uji p-value
p_values = {}

# Hitung p-value untuk setiap variabel numerik
for num_var in numerik:
    f_statistic, p_value =
stats.f_oneway(newTrain[newTrain[target_variable] ==
'normal'][num_var], newTrain[newTrain[target_variable] !=
'normal'][num_var])
    p_values[num_var] = p_value

# Hitung p-value untuk setiap variabel kategorikal
for cat_var in kategorikal:
    if cat_var != target_variable:
        contingency_table =
pd.crosstab(newTrain[target_variable], newTrain[cat_var])
        chi2, p_value, _, _ =
stats.chi2_contingency(contingency_table)
        p_values[cat_var] = p_value

# Menyaring variabel dengan p-value < 0.05
significant_variables = {var: p for var, p in
p_values.items() if p < 0.5}

# Menampilkan variabel-variabel yang signifikan beserta
nilai p-value
for var, p_value in significant_variables.items():
    print(f"{var}: p-value = {p_value}")
    
```

```

significant_variables = {var: p for var, p in
p_values.items() if p == 0}

for var, p_value in significant_variables.items():
    print(f"{var}")

significant_variables = {var: p for var, p in
p_values.items() if p > 0.5}

for var, p_value in significant_variables.items():
    print(f"{var}")

significant_variables = {var: p for var, p in
p_values.items() if 0 < p < 0.5}

for var, p_value in significant_variables.items():
    print(f"{var}")

newTrain['type_of_attack'].unique()

frequency_table =
dataTrain['type_of_attack'].value_counts()
print(frequency_table)

data = newTrain

columns_to_plot = [col for col in data.columns if col !=
'type_of_attack']

for column in columns_to_plot:
    plt.figure(figsize=(12, 6))
    if data[column].dtype == 'object':
        # CATEGORICAL buat count plot
        sns.countplot(x='type_of_attack', hue=column,
data=data)
        plt.xticks(rotation=90)
    else:
        # NUMERIK box plot
        sns.boxplot(x='type_of_attack', y=column,
data=data)
        plt.yscale('log') # Jika skala logaritmik
diperlukan

```



```
plt.title(f'{column} vs type_of_attack')
plt.show()

#pip install researchpy

## CRAMER V
import researchpy as rp

contingency_table = pd.crosstab(newTrain['type_of_attack'],
newTrain['protocol_type'])

# Hitung Cramer's V
cramer_v = rp.crosstab(newTrain['type_of_attack'],
newTrain['protocol_type'],
                        prop='cell')

# Tampilkan hasil
print(cramer_v)

contingency_table = pd.crosstab(newTrain['type_of_attack'],
newTrain['service'])

cramer_v = rp.crosstab(newTrain['type_of_attack'],
newTrain['service'],
                        prop='cell')

print(cramer_v)

contingency_table = pd.crosstab(newTrain['type_of_attack'],
newTrain['flag'])

cramer_v = rp.crosstab(newTrain['type_of_attack'],
newTrain['flag'],
                        prop='cell')

print(cramer_v)

"""# Modelling"""

# Modelling
from sklearn.ensemble import RandomForestClassifier
from sklearn.metrics import accuracy_score,
```





```

"srv_error_rate", "error_rate", "srv_error_rate",
"same_srv_rate", "dst_host_count", "dst_host_srv_count",
"dst_host_same_srv_rate", "dst_host_diff_srv_rate",
"dst_host_error_rate", "dst_host_srv_error_rate",
"dst_host_rerror_rate", "dst_host_srv_rerror_rate",
"protocol_type", "service", "flag"]

X2_train, X2_test, y2_train, y2_test = train_test_split(X2,
y, test_size=0.2)

rf2 = RandomForestClassifier()
rf2.fit(X2_train, y2_train)

y2_pred = rf2.predict(X2_test)

accuracy = accuracy_score(y2_test, y2_pred)
print("Accuracy:", accuracy)

"""Import Predictors Data"""

dataPredict = pd.read_csv("./Data_Prediction.csv", sep=";",
na_values=["*", 99999, "#NA"])

dataPredict['flag'] = le.fit_transform(dataPredict['flag'])
dataPredict['protocol_type'] =
le.fit_transform(dataPredict['protocol_type'])
dataPredict['service'] =
le.fit_transform(dataPredict['service'])

dataPredict.head()

id = dataPredict['id']
dataPredict.drop('id', axis=1, inplace=True)
predicted = rf1.predict(dataPredict)

predicted

predicted = le.inverse_transform(predicted)

predicted

dataPredict["prediction"]=predicted

```



DATA ANALYSIS COMPETITION 2023

INSTITUT TEKNOLOGI SEPULUH NOPEMBER

```
dataPredict["id"]=id  
  
dataPredict.to_csv("./prediction.csv", index=False)
```